

AXON-RFC-006 附录 B: easynet.web_app

由 agent 部署的全栈应用：技术设计、构建管线、Hub 路由与实现计划

胡思蓝
新加坡国立大学

草稿 v0.1, April 28, 2026

Abstract

本附录规定 `easynet.web_app`: 用于支撑完整现代 web 部署的 `state_type`。一个 agent (典型情形为 Claude Code) 在其 workspace 内撰写真实的 React/Vue/Next 风格前端项目, 通过构建产生静态 `dist/`, 该 `dist/` 与一个由 EasyNet ability 直接充当的后端一起被服务。Hub 扮演有状态版的 *nginx* 角色: 服务前端 bundle、把 `POST /api/<verb>` 翻译成 owner agent 上的 ability 调用、把 WebSocket 订阅代理到 ability stream, 并通过 `/__easynet/agent` 暴露同一应用对机器可执行的视图。本附录覆盖现代项目模型、canonical / runtime 状态拆分、长任务构建管线、API ability 映射、Hub 路由矩阵 (含适用于 SPA 的 CSP 与缓存规则)、让整个应用对邻居 agent 可发现的 `agent_view` 形态、在 `EasyNet-Cli` 中落地所需的五个 PR 切片, 以及一个端到端的 worked example——agent 用 Vite + React 写一个购物前端、声明 API ability、构建、发布、上线, 并被审计的全过程。

Contents

1	相对 pages 与 web 网络历史的位置	2
2	目标与非目标	2
2.1	目标	3
2.2	非目标	3
3	现代项目模型	3
3.1	workspace 中存放什么	3
3.2	<code>easynet.app.toml</code>	4
3.3	为什么 API 面必须声明而不是自动检测	5
4	状态机	5
4.1	<code>state_type</code> 与 <code>state_key</code>	5
4.2	规范字段	5
4.3	规范状态空间	6
4.4	Runtime 叠加状态空间	6
4.5	Transition 列表	7
5	构建管线	7
5.1	<code>webapp.build@v1</code> 步骤	7
5.2	TreeBlob 格式	8
5.3	构建执行器	8
6	Hub 路由模型	8
6.1	按状态划分的 Hub 行为矩阵	9
6.2	Action 分派	9
7	<code>agent_view</code> : 整应用可发现性	10

8 架构	12
8.1 模块拓扑 (EasyNet-Cli)	12
8.2 守护进程接线	12
8.3 Hub 接线	13
9 实现计划	13
9.1 P-B1: 构建执行器 + TreeBlob	13
9.2 P-B2: webapp.* 能力	14
9.3 P-B3: Hub 静态 + API 路由	14
9.4 P-B4: WebSocket 代理 + SPA fallback	14
9.5 P-B5: agent_view + meta 端点	15
10 端到端 worked example	15
10.1 准备	15
10.2 编辑项目	15
10.3 创建、构建、上线	15
10.4 浏览器拉取	16
10.5 浏览器侧 API 调用	16
10.6 邻居 agent 发现该应用	16
10.7 审计重放	17
11 暂缓至 RFC-006-B v2 的开放问题	17
11.1 覆盖检查	18

1 相对 pages 与 web 网络历史的位置

[推论] web 从一个静态文档媒介演变成完整应用平台。EasyNet 有状态 web 层走相同的弧，分两份附录：

附录	建模对象
A (easynet.page)	单一文档。作者写 HTML / Markdown；Hub 服务它。最简参考——单一规范状态、无 runtime 叠加、无构建步骤、无 API 面。
B (easynet.web_app)	完整应用。作者写真实前端项目，构建管线产生静态 bundle，应用的 API endpoint 即 EasyNet ability。Hub 在统一 URL 空间内服务前端 + API + WebSocket + agent_view。

两个都保留。Pages 仍是协议规范核最简化的验证；web_app 是第一个真正调用完整压力测试面 (markdown 配套 §B.1–B.8) 的状态类型。只交付 pages 而不交付 web_app 的实现，等于解决了有状态 EasyNet 简单的那一半；本附录规定的是难的那一半。

为什么这不是 nginx。 nginx 只服务一个静态目录、并把请求反向代理到独立部署的后端。后端对 nginx 是黑盒——它不理解后端状态，不知道后端有哪些 endpoint，也无法对一个通过结构化发现机制询问“调用方在这里能做什么”的邻居作答。本附录规定的 Hub 服务的字节与 nginx 一样，但其路由决策派生自 state object 的规范状态、其 API endpoint 是和其他每个 EasyNet 调用方共享同一注册表的、可内省的 ability，并且双视图契约让同一 URL 空间同时承载人类内容和对邻居 agent 可执行的整个应用地图。nginx 是 HTTP 管道；Hub 是恰好在外界讲 HTTP 的状态投影层。

2 目标与非目标

2.1 目标

[规范性] 一个符合本附录的实现：

1. 让 agent 在自己的 workspace 中用任意工具链（Vite、Next.js、Remix 等）撰写真实前端项目；EasyNet 不发明项目结构。
2. 提供长任务规范 `transition webapp.build`，运行项目声明的构建命令、把产出的 `dist/`（或等价物）以一份不可变的内容寻址 `bundle` 形式捕获，并按成功/失败推进 `canonical state`。
3. 提供 runtime 叠加 `transition (webapp.serve、webapp.stop)` 绑定本地端口并开始服务——关键点：不改 `canonical state`。
4. 在 Hub 上把 HTTP 请求按三条正交通道翻译为 EasyNet 操作：静态 `bundle` 拉取、API ability 调用、WebSocket ability 订阅。
5. 暴露单一的 `agent_view URL (/__easynet/agent)` 返回应用完整可发现性面：路由表、API endpoint、对 `transition` 的 `ability_surface`、构建元数据、回执历史指针、EAL 提示。
6. 上述全部能在五个 PR 切片 (§9) 中落地，每个切片可独立审阅。

2.2 非目标

[推论] 不在本附录范围：

- TLS 终结、HTTP/2 协商、多实例 Hub 集群。属于运维，不属于协议。
- 选定的前端框架。附录以 React/Vite 作 worked example 是因为 Claude Code 默认使用它；协议本身与框架无关。
- 需要每请求保持进程的服务端渲染。Phase 1 支持静态 + Hub 路由 API + WebSocket。需要每个请求都跑 Node.js 进程的 SSR 推迟到 RFC-006-B v2。
- 跨 realm 或跨节点的应用联邦。Phase 1：一个 `web_app` 属于唯一 agent、运行在唯一节点。
- 已认证人类 (`principal/human-auth`)。Phase 1 仅支持 `principal/human-anon`；应用自己设的 cookie 透传；EasyNet 不签发 cookie。

3 现代项目模型

3.1 workspace 中存放什么

[规范性] `state object` 在磁盘上的真值来源是一个普通的项目目录，归 agent 所有。具体：

```
<workspace>/web_apps/<slug>/
  easynet.app.toml      # EasyNet 侧的声明
  package.json          # agent 工具链所有
  package-lock.json     # 同上
  src/                  # 框架风格源代码
  public/
  vite.config.ts        # 或 next.config.js 等
  (可选) 框架需要的任何其他文件
```

EasyNet 只拥有 `easynet.app.toml`；其余属于框架。当 agent 运行 `webapp.build` 时，`easynet.app.toml` 中声明的命令在该目录中执行；它在所声明的 `dist_dir` 中产出的任何东西都会作为构建产物

被捕获。

3.2 easynet.app.toml

[规范性] EasyNet 用以理解 web_app 的唯一文件：

```
# easynet.app.toml —— EasyNet 编写并阅读的唯一文件。
# 项目其余部分属于框架。

[manifest]
name          = "shop"
description    = "由 agent 构建的小型陶瓷店。"
visibility     = "public"           # private | realm | public

[build]
command        = "npm install && npm run build"
dist_dir       = "dist"            # 相对项目根
build_timeout_secs = 600
node_version   = "20"              # 建议性；执行器可通过稳定镜像注册表
                                      # 钉版本

[serve]
# 调用 webapp.serve 时，运行时绑定一个端口并启动一个进程，其唯一职责
# 是为 Hub 反向代理暴露 API + WS；静态 bundle 从 blob 存储被服务，
# 不从这个进程被服务。
mode           = "static-only"
                # "static-only" : 无后端进程；API 完全由 Hub 发起的
                #                  ability 调用满足。Phase 1 的默认与
                #                  生产形态。
                # "process"       : 执行器启动后端进程；其端口出现在
                #                  runtime 叠加状态中。预留 v2。

[api]
# 前端\emph{允许}通过 Hub 的 /api/<verb> 路由调用的 ability。每条是
# owner agent 已注册的全限定 ability 名。Hub 会拒绝把 verb 不在此列表
# 的 /api/<verb> 请求翻译；这是 agent 已经显式授权的攻击面。
abilities = [
  "shop.list_items",
  "shop.checkout",
  "shop.order_status"
]

[routes]
# nginx 风格的路由规则。顺序重要。每条规则有 match 模式与 action。
rules = [
  { match = "exact:/",          action = "static:dist/index.html" },
  { match = "prefix:/assets/",  action = "static:dist/assets/" },
  { match = "regex:^(/api/([a-z][a-z0-9_]*)$)",
    action = "api:$1",
    methods = ["GET", "POST"] },
  { match = "prefix:/ws/",      action = "ws:$path" },
  { match = "exact:/__easynet/agent", action = "agent_view" },
  { match = "exact:/__easynet/health", action = "health" },
  { match = "exact:/__easynet/receipts", action = "receipts" },
  { match = "fallback",        action = "spa:dist/index.html" }
]

[csp]
```

```
# 收紧或放宽默认值。默认值适用于普通 SPA；需要 'unsafe-inline' 的
# 框架必须显式选择。
extra_script_src = [] # 例如内联引导脚本的 ['sha256-...']
extra_style_src = ['unsafe-inline']
extra_connect_src= ["self"]
```

文件小、声明式，且是参与 canonical state 哈希的唯一对象（与源树和构建产物哈希并列）。最小应用三个段（[manifest]、[build]、[api]）即可；routes 默认走上面的 SPA 标准布局。

3.3 为什么 API 面必须声明而不是自动检测

[规范性] 一个 web_app 的 API 面是声明的，不是发现的。当 <verb> 不在 [api].abilities 里时，Hub 拒绝把 /api/<verb> 翻译成调用。两条理由：

- **审计清晰。** 回执日志精准记录公开 web_app 暴露的 ability。审计者在 canonical state 里看到该面，而不是在运行时内省。
- **攻击面收敛。** owner agent 可能托管很多 ability；只有显式列出的能从公开 Hub 触达。agent 注册的内部用 ability 在 web 上不可见。

不变量 — WB-INV-1

当 <verb> 不在 web_app 当前 Built canonical state 的 api_abilities 列表中时，Hub 必须拒绝把 /api/<verb> 翻译为 ability 调用。拒绝必须返 HTTP 404（不是 403）以避免确认内部 ability 的存在。这是身份层 TR-INV-12 建立的信任边界纪律在应用层的实例化。

4 状态机

4.1 state_type 与 state_key

[规范性]

state_type: easynet.web_app

state_key:

(realm: string, owner_agent_uri: string, app_slug: string)

realm 逻辑命名空间；与 pages 同惯例。

owner_agent_uri 规范权限。即为本 app 每个 CanonicalReceipt 签名的 agent。按主文 §1.3 嵌在 key 内（不是字段）。

app_slug owner 命名空间内的 kebab-case 标识符
(^[a-z0-9][a-z0-9-]{0,62}[a-z0-9]\$)。

URA 形态：

easynet:///r/<realm>/agent/<owner>/web_app/<slug>

4.2 规范字段

[规范性] 按 CSH-INV-2（主文 §2.5）大体积值都按内容哈希持有：

```
canonical_fields(web_app) = {
    manifest_hash:    bytes(32), // 规范化后 [manifest] 段的
                        // SHA-256 (name, description,
                        // visibility)
    build_config_hash: bytes(32), // 规范化后 [build] 段
```

```

api_surface_hash:    bytes(32), // 规范化后 [api].abilities
                        //      (已排序)
routes_manifest_hash:bytes(32), // 规范化后 [routes].rules
                        //      (按声明顺序)
csp_hash:            bytes(32), // 规范化后 [csp] 段
source_snapshot_hash:bytes(32), // 最近一次构建源树的哈希
                        //      首次构建前为 null
build_artifact_hash: bytes(32), // 上次成功构建产生的 dist 树
                        //      哈希。首次 Built 前为 null。
                        //      delete 时清零。
state_value:         string,    // §4.3
visibility:           string,    // 镜像 manifest.visibility
                        //      便于快速 scope check
version:              uint64
}

```

canonicity 规则。上述每个字段在 schema 中均标 `canonical:true`；schema 驱动的投影 π 恰按上述 key 产出投影。

4.3 规范状态空间

state_value	含义
Created	已注册 easynet.app.toml；尚未尝试构建。
Building	一次 webapp.build 在进行中。瞬态。仅通过 OperationalReceipt 可见；从不作为终态 canonical state。
Built	已存在至少一次成功构建。build_artifact_hash 非空。可被 Served。
Failed	最近一次构建失败。owner 可迭代后重试。
Deleted	终态；build_artifact_hash 清零，blob 引用计数递减。

4.4 Runtime 叠加状态空间

runtime	含义
Stopped	此 app 没有附着的服务叠加。
Starting	webapp.serve 已被调用；Hub 正在把 slug 绑定到其路由。
Running	Hub 正主动为此 app 翻译 <code><slug>/...</code> 请求。
Crashed	Hub 丢失了映射（进程重启、配置重载错误等）；canonical state 未被触动。可通过 webapp.serve 恢复。

canonical $\times$ runtime 交叉积矩阵在结构上与 markdown 配套 §B.4.3 完全一致：仅 Built 允许非 Stopped 的 runtime 叠加。

4.5 Transition 列表

Transition	类	说明
webapp.create@v1	canonical	$\emptyset \rightarrow \text{Created}$ 。铸造 <code>state_key</code> ，落 <code>manifest/build_config/routes/api/csp</code> 哈希。
webapp.update_config@v1	canonical	<code>Created/Built/Failed</code> $\rightarrow$ 同。替换声明配置 (<code>manifest, build, api, routes, csp</code>)。不触发构建。
webapp.build@v1	canonical	<code>Created/Built/Failed</code> $\rightarrow$ <code>Building</code> (瞬态) $\rightarrow$ <code>Built/Failed</code> 。长任务。见 §5。
webapp.delete@v1	canonical	$*$ $\rightarrow$ <code>Deleted</code> 。隐式触发 <code>webapp.stop</code> 副作用。
webapp.serve@v1	operational	<code>Stopped/Crashed</code> $\rightarrow$ <code>Starting</code> $\rightarrow$ <code>Running</code> 。要求 <code>canonical = Built</code> 。
webapp.stop@v1	operational	<code>Starting/Running/Crashed</code> $\rightarrow$ <code>Stopped</code> 。
webapp.get@v1	query	返回 <code>canonical + runtime</code> 快照。
webapp.list@v1	query	列出某 agent 拥有的 <code>web_app</code> 。

5 构建管线

5.1 webapp.build@v1 步骤

[规范性] webapp.build 时执行器：

1. 读取 `easynet.app.toml` 并校验。
2. 计算 `source_snapshot_hash`：项目目录的 Merkle 哈希（排除 `node_modules/`、`dist/`、`.git/` 以及框架 `.gitignore` 排除的任何路径）。哈希取自一个排序后的 (`relative_path, sha256(file_bytes)`) 对列表，已 JCS 规范化。
3. 发出 `build.started OperationalReceipt`，含源快照哈希、声明命令、预期超时。非规范。
4. 在项目目录内 spawn 构建命令。`stdout / stderr` 被三通到流式日志；按周期发出 `build.progress OperationalReceipt`，含日志累积哈希。可丢失。
5. 退出时：
 - 状态码 0 且 `dist_dir` 非空：遍历 `dist_dir`；按 (`relative_path, sha256(file_bytes)`) 对计算 `build_artifact_hash` Merkle 哈希；把整棵 `dist` 树作为单个 `TreeBlob`（一份按 JCS 规范化的 `manifest`，列出文件路径与各自 `blob` 哈希；文件 `blob` 作为普通内容寻址 `blob` 单独存储）写到 `blob` 存储。
 - 状态码非零或 `dist_dir` 空：构建失败原因 `payload`（最后 64KB 日志 + 退出状态码）并存其哈希。
6. 发出唯一一条 `CanonicalReceipt`：成功为 `build.completed`，失败为 `build.failed`。这是该 transition 的唯一规范推进。`post_state_hash` 反映 `Built` 或 `Failed`；`state_version + 1`。

canonicity 纪律。瞬态 `Building`、流式进度、构建工具 `stdout` 都是观测。它们绝不进入规范哈希。仅有规范回执的验证者看到应用一步从 `Created` $\rightarrow$ `Built`（或 `Failed`）——这正确：规范事实是产物，不是过程。

5.2 TreeBlob 格式

[规范性] TreeBlob 是目录树的内容寻址表示。它包含：

```
TreeBlob (JCS 规范化 JSON)
{
  "version": 1,
  "entries": [
    { "path": "index.html",
      "size": 2148,
      "blob": "<file 字节的 sha256>",
      "mode": "0644" },
    { "path": "assets/index-DfX9k.js",
      "size": 187321,
      "blob": "<sha256>",
      "mode": "0644" },
    ...
  ]
}
```

树的哈希是 `sha256(jcs(TreeBlob))`；这就是 `build_artifact_hash`。每个独立文件也是内容寻址 blob。Hub 通过先在 TreeBlob 中按 `path lookup`、再 `blob.get(<file-blob-hash>)` 拉取文件；多一次间接但让 blob 存储在多次构建间、跨多个应用间去重相同字节。

为什么不直接用 git tree hash。 git 的 tree hash 也是内容寻址，但用 git 特有方式编码 mode 位与 entry 类型。我们选 JCS 扁平格式以让哈希函数与 EasyNet 其他地方完全一致（SHA-256 over JCS 字节），避免出现第二种规范化机制。

5.3 构建执行器

[规范性] Phase 1 交付一个本地构建执行器：构建在拥有 agent 的 EasyNet 守护进程的子进程中运行。owner 直接为产生的 CanonicalReceipt 签名。委托构建（源在 agent A，构建在 agent B 的硬件运行）属于未来：协议形态已就绪（主文 §5.3），但执行器尚未实现。

待决策

待决策：构建沙箱。 构建子进程具备 EasyNet 守护进程的进程权限。恶意构建脚本可能读取守护进程托管的其他 agent 状态。Phase 1 缓解：构建在项目目录内运行，限制 PATH，仅继承 `NODE_VERSION` 与 `HOME` 等少量环境变量，且更严的 `ulimit`。完整沙箱（`seccomp / pledge / Docker`）属于 RFC-006-B v2。

6 Hub 路由模型

Hub 绑定单端口（Phase 1 默认 127.0.0.1:8787）。每个进入的 HTTP 请求按七步处理：

1. **caller 铸造。**按主文 §5.4 构 `envelope:caller = principal/human-anon/<ip-hash>/<sid>`；`caller_context` 按主文 §3.6 / TR-INV-13。
2. **path 解析。** path 是其中一种：
 - `/<realm>/<owner>/<slug>/...` —— 规范形态。
 - 配置好的别名域名下任意路径（如 `shop.alice.example` 映射到一组特定 (`realm`, `owner`, `slug`)）。

3. **state 解析**。在 state 存储中查 web_app。若不在 Built，按 §6.1 返。若 visibility 拒绝 caller，返 404。
4. **route 匹配**。按声明顺序在 web_app 的 [routes].rules 中跑 path 尾段直到一条命中。
5. **action 分派**。按命中规则的 action 分派 (static / api / ws / agent_view / health / receipts / spa fallback)。见 §6.2。
6. **响应构建**。按 action 设置 CSP、ETag、缓存头、content-type。
7. **OperationalReceipt**。发出一条 OperationalReceipt，含 caller、caller_context、命中规则、action、响应状态。非规范。

6.1 按状态划分的 Hub 行为矩阵

Canonical	Runtime	Hub 响应
Created	*	503 "未构建"。Retry-After: 60。
Building	*	503 + 通过 SSE 流式发送 build.progress 回执的小进度页。
Built	Stopped	503 "未在服务"。
Built	Starting	503 "启动中"，Retry-After: 2。
Built	Running	完整路由矩阵如下。
Built	Crashed	503 "可恢复失败" + reason hash; Retry-After: 5。
Failed	*	503 "上次构建失败" + reason hash。
Deleted	*	404。

6.2 Action 分派

static:<path> 在 web_app 的 build_artifact_hash TreeBlob 内解析 <path>。拉取 file blob。响应：

```
HTTP/1.1 200 OK
Content-Type: <由扩展名推断；走允许列表映射>
Content-Length: <size>
ETag: "<base64url(blob_hash[..16])>"
Cache-Control: public, max-age=31536000, immutable
                  # 仅对 /assets/* (带哈希文件名)；
                  # 否则: max-age=60, must-revalidate
Content-Security-Policy: <由 app 的 csp_hash 构建>
Referrer-Policy: no-referrer
X-Content-Type-Options: nosniff
```

immutable 缓存策略对带哈希文件名的资源安全 (Vite/Webpack 输出的资源文件名内嵌内容哈希)；index.html 等无哈希文件用短的 must-revalidate 窗口。

api:<verb> <verb> 是 route 规则的 regex 捕获。Hub：

1. 校验 <owner>.<verb> 在 api_surface_hash 对应展开的能力列表中。拒绝：HTTP 404 (WB-INV-1)。
2. 把请求 body (Phase 1 大小上限 1 MiB) 作 JSON 读取。该 JSON 即 ability 的 args，无 envelope 包裹。GET 请求用 query string；Hub 把 query string 解析为对象。
3. 用此 caller、caller_context、解析后的 args 构 EasyNet invocation。

4. 调用 `<owner>.<verb>`。

5. 把结果映射到 HTTP:

- Succeeded: 200, body = 回执 result body, content-type application/json。
- 带结构化错误的 Failed 回执: 校验类 400, 否则 500; body = { error_code, error_message, receipt_id }。
- 超时: 504; 含运维超时回执的 receipt_id。

幂等性。 GET 与 POST 都允许; 幂等性是 ability 自己的责任, 不归 Hub。Hub 不强制 GET 不产 CanonicalReceipt——它是 EasyNet ability, 按其语义决定是否要发。前端选动词; ability 作者在 schema 中记录幂等约定。

ws:<path> Hub 把连接升级为 WebSocket 并绑到一个 ability stream (已有的 EasyNet stream 订阅机制)。客户端首条 WebSocket 消息必须是 JSON { "ability": "<verb>", "args": { ... } }; 之后 Hub 打开 `<owner>.<verb>` 流并把每个 emit 事件作为 WebSocket 文本帧转发。首条之后客户端消息回填给 stream 的输入通道。WebSocket 关闭映射为 stream 关闭。

agent_view 返回 §7 规定的 JSON。

health 返回 200 {"state":"...","runtime":"..."}。

receipts 返回该 state_key 的规范回执历史, 可由 ?from=<version>&to=<version> 约束。

spa:<fallback-path> 作为最后的 fallback 规则。等价于 static:<fallback-path> 但带 Cache-Control: no-cache, 让 SPA 外壳始终新鲜, 而其加载的 hashed 资源不可变。

7 agent_view: 整应用可发现性

[规范性] 一个 Built/Running web_app 的 agent_view:

```
GET /<realm>/<owner>/<slug>/__easynet/agent

200 OK
Content-Type: application/json

{
  "state_type": "easynet.web_app",
  "state_key": { "realm":"default","owner":"alice","slug":"shop" },
  "ura": "easynet:///r/default/agent/alice/web_app/shop",

  "canonical": {
    "state": "Built",
    "manifest": { "name":"shop",
                  "description":"由 agent 构建的小型陶瓷店。",
                  "visibility":"public" },
    "build_artifact_hash":"0x4d28ee17...",
    "version": 7,
    "etag": "TSjuFw0pAvjpAQ=="
  },

  "runtime": {
    "state": "Running",
```

```

    "started_at": "2026-04-28T09:11:02Z",
    "host":       "https://shop.alice.example"
                  // 未配置别名域名时为 null
  },

  "ability_surface": [ // 当前可执行的 transition
    "webapp.update_config@v1",
    "webapp.build@v1",
    "webapp.stop@v1",
    "webapp.delete@v1"
  ],

  "api": { // 前端可经 /api/* 调用的内容
    "endpoints": [
      { "verb": "shop.list_items",
        "http": "GET /api/list_items",
        "schema_url": "/__easynet/schema/shop.list_items" },
      { "verb": "shop.checkout",
        "http": "POST /api/checkout",
        "schema_url": "/__easynet/schema/shop.checkout" },
      { "verb": "shop.order_status",
        "http": "GET /api/order_status",
        "schema_url": "/__easynet/schema/shop.order_status" }
    ]
  },

  "routes": [
    { "match": "exact:/", "action": "static:dist/index.html" },
    { "match": "prefix:/assets/", "action": "static:dist/assets/" },
    { "match": "regex:~/api/...", "action": "api:$1" },
    { "match": "prefix:/ws/", "action": "ws:$path" },
    { "match": "fallback", "action": "spa:dist/index.html" }
  ],

  "build_meta": {
    "command": "npm install && npm run build",
    "dist_dir": "dist",
    "node_version": "20",
    "last_build_at": "2026-04-28T09:09:51Z",
    "build_duration_s": 38.2,
    "framework_hint": "vite-react" // 尽力嗅探
  },

  "history": {
    "versions": 7,
    "latest_canonical_receipt_id": "r_c3d4...",
    "receipt_log_ura":
      "easynet:///r/default/agent/alice/web_app/shop/receipts"
  },

  "eal_hint":
    "call shop.checkout on alice with { item_id: \"mug-12oz\", qty: 1 } timeout 30"
}

```

为什么这是 killer feature。邻居 agent 读到此 URL，一份文档里就拿到了它需要的一切：

- 知道这是什么应用、它处于何种状态；

- 精确知道前端可以调用哪些 API verb (含路径、方法、schema 指针)；
- 知道哪些 transition 推进该应用 (ability_surface)；
- 既能直接经 EasyNet 调用任何一个 verb，也能经公开 /api/<verb> 调用——两条路径触达同一个 ability；
- 端到端重放应用历史并在密码学上链回到 owner。

这是 agent-native web。传统 web 应用对人暴露 HTML、对程序暴露文档化 endpoint；邻居程序需要文档、OpenAPI 规约、SDK 与信任。本视图把这四样合并。

8 架构

8.1 模块拓扑 (EasyNet-Cli)

src/runtime/state/	[与 pages 共享, P-A1]
src/runtime/blob_store/ tree.rs	[与 pages 共享, P-A1] ← 新: TreeBlob put/get、 per-path lookup
src/runtime/agents/webapp_ability.rs	← 新 (P-B2) register: create / build / update_config / serve / stop / delete / get / list
src/runtime/build_executor/ mod.rs source_snapshot.rs spawn.rs artifact_capture.rs	← 新 (P-B1) 长任务规范 transition 驱动 项目目录的 Merkle 哈希 子进程 + 日志流 dist 树 -> TreeBlob
src/runtime/views/webapp/ agent_view.rs health.rs receipts.rs	← 新 (P-B5) S6 的 JSON 投影
src/runtime/hub/ router.rs static_handler.rs api_handler.rs ws_handler.rs	[扩展 P-A4 的 pages Hub] ← 扩展以读取 routes_manifest ← 新 (P-B3): TreeBlob 拉取 ← 新 (P-B3): /api/<verb> 翻译 ← 新 (P-B4): /ws/* 升级与 stream 绑定

8.2 守护进程接线

启动时守护进程在 pages 接线之上加：

1. 在 \$EASYNET_HOME/build-cache/ 打开构建执行器工作目录。
2. 给每个 agent 注册 webapp_ability。
3. 对每个规范状态为 Building 的 web_app (即守护进程重启打断了一次构建)：标 Failed 并把 reason 设为“守护进程在构建途中重启”。(按 TR-INV-1 纪律：仅终结规范回执推进状态；中断的构建从未触达终结，因此规范状态回滚到 Building 之前的状态——Created 或 Failed。重置为 Failed 是新 transition，不是回滚。)

8.3 Hub 接线

Hub 二进制 easynet-hub 在 P-B3/P-B4 中加上 `static_handler / api_handler / ws_handler`, 与 P-A4 的 `page handler` 并列。其顶层分派为:

```
fn handle(req: HttpRequest) -> HttpResponse {
  let envelope = build_envelope(&req);           // §5.7 PA
  let (realm, owner, slug, tail) = parse_path(&req.uri)?;
  let kind = state_store.classify(realm, owner, slug)?; // page or web_app

  match kind {
    Kind::Page    => pages_dispatch(envelope, owner, slug, tail),
    Kind::WebApp  => webapp_dispatch(envelope, owner, slug, tail),
    Kind::Unknown => http_404(),
  }
}
```

Hub 即使同时服务 `pages` 与 `web_app` 也是单一二进制。path 方案仅按 `state` 存储中 (`owner`, `slug`) 注册的内容来区分两者——URI 形态完全相同, 因为两者都是同一 `agent` 命名空间下的 `state object`。

9 实现计划

五个 PR 切片, 可顺序化。切片 P-B1 基于 P-A1 (`pages` 附录的 `blob` 存储与 `state` 存储骨架); P-B3/B4 基于 P-A4 (基本 Hub 二进制)。

9.1 P-B1: 构建执行器 + TreeBlob

新增文件。src/runtime/build_executor/ (4 个模块)、src/runtime/blob_store/tree.rs。
公开 API。

```
pub trait BuildExecutor: Send + Sync {
  fn execute(
    &self,
    project_dir: &Path,
    config: &BuildConfig,
    progress: ProgressSink, // 通往 OperationalReceipt 的 SSE 通道
  ) -> Result<BuildOutcome>;
}

pub enum BuildOutcome {
  Success { artifact_hash: [u8; 32], tree_blob: TreeBlob, log_blob: [u8; 32] },
  Failure { reason_hash: [u8; 32], log_blob: [u8; 32], exit_status: i32 },
}

pub fn source_snapshot_hash(project_dir: &Path, ignore: &[Pattern])
  -> Result<[u8; 32]>;

pub fn capture_dist(dist_dir: &Path, blob: &dyn BlobStore)
  -> Result<(TreeBlob, [u8; 32])>;
```

测试。

- 快照确定性: 同一棵树两次快照得相同哈希, 与文件遍历顺序无关。
- 构建烟测: 一个微型 shell 脚本”构建器” (`mkdir dist && echo hi > dist/index.html`) 端到端驱动执行器; 断言 `TreeBlob` 往返且产物哈希可确定性重算。

- 守护进程在构建中崩溃: 启动一个 sleep 的构建; SIGKILL 守护进程; 启动后断言 web_app 的规范状态被重置为 Failed 并附文档化 reason。

9.2 P-B2: webapp.* 能力

新增文件。 src/runtime/agents/webapp_ability.rs。

行为。 §4.5 中每个 transition 都被实现, 包括长任务构建 (按 TR-INV-1 流式发出进度 OperationalReceipt 并每次尝试发出且仅发出一条终结 CanonicalReceipt)。

测试。

- 往返 create/update_config/build/serve/stop/delete。
- 拒绝从 Created 出发的 webapp.serve (规范前置: Built)。
- 当 <verb> 不在 api_surface 时拒绝 api/<verb> 翻译。
- 乐观并发: 两个针对同一 web_app 的 webapp.update_config 并发尝试; 一个赢, 一个被 ETag 不一致拒绝。
- 构建进度回执可丢失: 随机丢一些; 重放仍达相同终结规范状态。

9.3 P-B3: Hub 静态 + API 路由

新增文件。 src/runtime/hub/static_handler.rs、src/runtime/hub/api_handler.rs; 扩展 src/runtime/hub/router.rs 以读取 routes_manifest。

行为。

1. 静态处理器: TreeBlob path lookup、blob 拉取、扩展名 MIME 检测 (允许列表)、由 csp_hash 构 CSP, 对 /assets/* 用 immutable 缓存。
2. API 处理器: route regex 捕获 verb、按 api_surface 校验、把 JSON body 或 query string 解析为 args、调用 <owner>.<verb>、映射结果。

测试。

- 集成: 在临时目录里启动守护进程 + Hub, 构建一个 fixture web_app, 其 api.abilities 含已注册的 shop.echo (回填 args); POST /api/echo {"x":1} 得 200 {"x":1}。
- 未授权 verb: POST /api/secret 即使 <owner>.secret 已注册但不在 api_surface 中也返 404。
- 静态 immutable: 对 /assets/ 路径带 If-None-Match 两次 GET; 第二次响应为 304。

9.4 P-B4: WebSocket 代理 + SPA fallback

新增文件。 src/runtime/hub/ws_handler.rs。

行为。

1. /ws/* 上 WS 升级; 首条文本消息必须是 JSON {ability,args}; 绑到 ability stream; 转发后续文本帧。
2. SPA fallback: 若 static 或其他规则均不命中, 用 no-cache 服务 dist/index.html。

测试。 WS 流: fixture shop.tick 每 100ms 流式发出 {tick:N}; 客户端通过 /ws/tick 订阅; 断言 600ms 内收 5 帧; 客户端关 WS 后 500ms 内守护进程的流订阅关闭。

9.5 P-B5: agent_view + meta 端点

新增文件。src/runtime/views/webapp/ (3 模块); 对 /__easynet/* 前缀做轻微路由扩展。行为。

1. /__easynet/agent 返回 §6 的 JSON。
2. /__easynet/health 仅返规范与 runtime 状态。
3. /__easynet/receipts 按 version 顺序返回规范回执, 可由 query param 约束。
4. /__easynet/schema/<verb> 返回该 ability 的输入/ 输出 schema。

测试。

- 重放: 客户端拉 /__easynet/receipts, 按每条回执重构规范状态, 并计算与 agent_view 的 etag 匹配的 canonical_state_hash。
- Schema: agent_view 的 api.endpoints[*].schema_url 可解析到真实 JSON schema, 且该 schema 不拒绝 API 处理器接受的 args。

10 端到端 worked example

10.1 准备

```
$ easynet agent add alice --type claude-code
$ easynet-daemon &
$ easynet-hub --bind 127.0.0.1:8787 &
```

10.2 编辑项目

Claude Code 以 alice 身份在 alice 的 workspace 中:

```
$ cd $EASYNET_HOME/workspaces/alice
$ npm create vite@latest web_apps/shop -- --template react-ts
$ cd web_apps/shop
$ # ... 编辑 src/App.tsx, 写产品列表与"结账"按钮
$ # ... 写 §3.2 中的 easynet.app.toml
```

它通过 EasyNet 能力脚手架 (不在本附录范围内; 见"ability authoring" 附录) 注册 API ability:

```
$ easynet ability new shop.list_items --description "列出商品" --lang sh
$ easynet ability new shop.checkout --description "下单" --lang sh
$ easynet ability deploy abilities/shop.list_items --node alice
$ easynet ability deploy abilities/shop.checkout --node alice
```

10.3 创建、构建、上线

```
$ easynet ability invoke alice.webapp.create --args '{
  "slug":"shop",
  "config_dir":"web_apps/shop"
}'
{ "state":"Created", "version":1, "canonical_state_hash":"0xa3..." }

$ easynet ability invoke alice.webapp.build --args '{ "slug":"shop" }' \
  --stream
[build.started ] command="npm install && npm run build"
```



```
[build.progress ] log_hash=0x12... 8% installed
[build.progress ] log_hash=0x13... 47% bundled
[build.progress ] log_hash=0x14... 92% optimised
[build.completed] artifact_hash=0x4d28ee17... duration=38.2s
{ "state": "Built", "version": 2,
  "build_artifact_hash": "0x4d28ee17..." }

$ easynet ability invoke alice.webapp.serve --args '{ "slug": "shop" }'
{ "runtime_state": "Running" }
```

10.4 浏览器拉取

```
$ curl -i http://127.0.0.1:8787/default/alice/shop/

HTTP/1.1 200 OK
Content-Type: text/html
ETag: "TSjuFwOpAvjpAQ=="
Cache-Control: max-age=60, must-revalidate
Content-Security-Policy: default-src 'self'; script-src 'self'; ...
Content-Length: 1247

<!doctype html><html><head>...<script type="module" src="/assets/index-DfX9k.js">...
```

随后浏览器拉 `/assets/index-DfX9k.js` —— `public, max-age=31536000, immutable`, 从 blob 存储服务。SPA 渲染、列出商品, 用户点 Buy。

10.5 浏览器侧 API 调用

```
POST /default/alice/shop/api/checkout HTTP/1.1
Content-Type: application/json
Cookie: easynet_session=sess-abc

{ "item_id": "mug-12oz", "qty": 1 }

HTTP/1.1 200 OK
Content-Type: application/json

{ "order_id": "ord_77a3f", "eta": "2026-04-29" }
```

Hub:

1. 铸造 `caller=easynet:///principal/human-anon/9c4e.../sess-abc`。
2. 校验 `checkout` 在 `api_surface` 中 (是)。
3. 用解析后 `body` 作 `args` 调用 `alice.shop.checkout`。
4. 把回执 `result body` 返给浏览器。

10.6 邻居 agent 发现该应用

```
$ curl http://127.0.0.1:8787/default/alice/shop/__easynet/agent | jq .api
{
  "endpoints": [
    { "verb": "shop.list_items", "http": "GET /api/list_items",
      "schema_url": "/__easynet/schema/shop.list_items" },
```

```
{ "verb": "shop.checkout", "http": "POST /api/checkout",
  "schema_url": "/_easynet/schema/shop.checkout" }
]
```

邻居 agent——比如运行在另一个节点上的 bob——拉取该视图，读 `shop.checkout` 的 `schema`，然后通过 EasyNet 总线（跨节点）或公开 `/api/checkout`（同一 Hub）触发该 ability。两条路径触达同一 ability handler，产生同种 CanonicalReceipt。

10.7 审计重放

`shop web_app` 的历史显示三条 CanonicalReceipt(create、build、build)：每条都带 `post_state_hash` 与 `owner_signature`。浏览器驱动的 checkout 触发了一条针对完全不同 `state_type` 的 CanonicalReceipt（一个由 alice 或 alice 的订单服务拥有的 order state object）；该回执也链回 alice 的签名密钥。同时阅读两条链的邻居可逐字节重构：

- 当 checkout 发生时，page 处于哪个构建产物；
- 那个构建声明了哪些 API endpoint；
- checkout 请求触达 `shop.checkout` 第 K 版并产出订单 `ord_77a3f`；
- 上述全部由 alice 签名。

这是传统 web 决然讲不出的审计故事。今天给 shop 服务的 CDN、明天的 A/B 重写、第三天的带外订单处理，把真相切成三块；`web_app` 把它合并为一条签名回执链。

11 暂缓至 RFC-006-B v2 的开放问题

1. **需要持续进程的服务端渲染。** Phase 1 支持 `serve.mode = static-only`。需要 Node.js 进程响应每个请求的框架（Next.js SSR、Remix loader）需要 `serve.mode = process`：执行器绑定端口、运行进程、Hub 反向代理。runtime 叠加状态那时会承载 `bound_port`、`process_id`、`health_endpoint`；进程崩溃把 runtime 转入 Crashed 而不触动 canonical (TR-INV-5 已要求)。
2. **跨节点 Hub。** A 节点拥有的 `web_app` 由 B 节点上的 Hub 服务。Hub 升级为联邦客户端；API 处理器经联邦而非本地调用 `<owner>.<verb>`。
3. **已签资源 bundle 用于 SRI。** TreeBlob 已经按文件提供哈希。把这些哈希嵌为 `index.html` 的 Subresource Integrity 属性是一个直白的 post-build 阶段；Phase 1 把 SRI 管线留给框架自身的插件。
4. **构建沙箱。** 见 §5 的待决策。
5. **多版本路由。** 用户希望 `/v7/...` 服务构建版本 7，而 `/...` 服务最新版。今天每个 slug 在规范状态里只能有单版本。v2 可能引入 `webapp.tag transition` 把版本与可路由别名关联。

11.1 覆盖检查

主文规则	由谁兑现	位置
SO-INV-1 (key 不可变)	state_key 解析器拒绝 key 更新	§4
CSH-INV-1 (hash 取自 projection)	schema 驱动的 π 、JCS、SHA-256	§4
CSH-INV-2 (内容外置为 blob)	TreeBlob + 单文件 blob; 规范状态只持哈希	§5
RC-INV-1 (receipt 绑定)	每条 webapp 回执均带 (state_key, transition_id, attempt_id)	§4
RC-INV-2 (乐观并发)	在 update_config 上做 ETag 检查	§9 P-B2
TA-INV-1 (必须带 transition_id)	每个 webapp.<verb>@v1 字符串均被校验	§4
VP-INV-1 (ability_surface 来自 state)	由 state_value 派生 ability_surface	§7
TR-INV-1 (长任务终结)	构建只发恰一条终结 CanonicalReceipt	§5
TR-INV-2 (progress 不进规范)	build.progress 是 OperationalReceipt	§5
TR-INV-5 (runtime 叠加独立)	runtime 崩溃不触动 canonical	§4
TR-INV-7 (multi-view 一等公民)	human (HTML+API+WS) 与 agent_view 都已实现	§6, §7
TR-INV-8 (cross-state-space 矩阵)	canonical $\times$ runtime 准入被强制	§4
TR-INV-11 (URA-receipt 可追踪)	/__easynet/receipts + state_key 索引	§10
TR-INV-12 (principal/human-anon)	Hub 在每个请求上铸造该 URI	§6
TR-INV-13 (caller_context)	由请求头填充; 记录到 operational receipt	§6, §10
WB-INV-1 (api_surface 强制)	Hub 拒绝未列出 verb 并返 404	§3

结语。 pages 是简单的一半: 单一文档作为规范状态, 无构建、无 runtime 叠加、无 API。web_app 是难的一半: 长任务规范 transition、runtime 叠加、声明式 API 面、完整 Hub 路由矩阵、WebSocket。落地 P-B1 至 P-B5 完成有状态 EasyNet web 层 Phase 1, 并以工业级形式落地 agent-native web。

历史类比: 早期 web 是静态文档 (*easynet.page*); 现代 web 是应用 (*easynet.web_app*)。EasyNet 规约有意走相同的弧。先 *pages*、后 *web_app*, 每一层都为下一层将要倚仗的协议原语作背书。